GEMINI AGENT PLATFORM · A 429-SURVIVAL CAPACITY LADDER

You don’t choose a consumption option. You order them.

When a request comes back `RESOURCE_EXHAUSTED`, the reflex is to ask the same pool the same question again. A **capacity ladder** asks somewhere else, changing exactly one variable per step — consumption option, model or endpoint — until something answers. **No Provisioned Throughput order required.**

SIX STEPS, ONE VARIABLE EACH — THE DEFAULT, COST-EFFECTIVE LADDER

STEP	CONSUMPTION OPTION	MODEL	ENDPOINT	WHAT MOVED
1 `flex`	Flex PayGo	`gemini-3.7-flash`	`global`	—
2 `standard`	**Standard PayGo**	`gemini-3.7-flash`	`global`	**tier**
3 `priority`	**Priority PayGo**	`gemini-3.7-flash`	`global`	**tier**
4 `alternative_model`	Standard PayGo	**`gemini-3.6-flash`**	`global`	**model**
5 `multi_region_us`	Standard PayGo	`gemini-3.6-flash`	**`us`**	**endpoint**
6 `multi_region_eu`	Standard PayGo	`gemini-3.6-flash`	**`eu`**	**endpoint**

Each step changes exactly one variable — consumption option, model or endpoint.

- **Steps 1–3** vary only the **consumption option**: Flex → Standard → Priority.
- **Step 4** changes only the **model**.
- **Steps 5–6** change only the **endpoint**.

CHOOSING A LADDER, OR MAKING YOUR OWN

PRESET	SHAPE	FOR
`play_smart_default()`	Flex → Standard → Priority → alternative → us → eu — the ladder above	Latency-tolerant, agentic
`latency_first()`	Priority on three models	Interactive, user-facing

Same engine, different order. A ladder is **data, not control flow** — reorder the list, or write your own in a dozen lines, and your cost and latency posture changes without touching client code.

THE 60-SECOND BET

Step 1 caps Flex at 60 seconds, one attempt, no retry. It is not a fallback — it is a **priced wager**: Flex is a published **50% discount**, so the step spends up to a minute of latency for a coin-flip at half price. The cap is the trick: **Flex’s default timeout is ten minutes** — right for the offline work it is sold for, a stall as a synchronous first hop.

90 **Tests that ship with it, offline and online** — exercise the ladder and the Gemini consumption options against **your own Google Cloud project**. The 429 is real and free to summon: every attempt recorded, option requested beside option granted.

Read the article

The ladder step by step, the two headers, the 60-second Flex bet, and all 21 traversals with the attempt trails.

Gemini Play Smart — a 429-survival capacity ladder →

Call the ladder instead of calling Gemini directly

Clone it — two imports and one object, and the same call you already make.

```
from play_smart import CapacityLadder, play_smart_default
ladder = CapacityLadder(play_smart_default(),
                        project="my-project")
result = ladder.generate_content("Summarise this contract.")

print(result.text)
print(result.step_used)  # 'flex', or wherever it landed
print(result.table())    # every hop, asked vs granted
```

github.com/eilonbar/gemini-play-smart →

Eilon Bar AI Customer Engineer, Google Cloud medium.com/@eilonbar github.com/eilonbar/gemini-play-smart